Space-Runner: A Game for the Basys3 FPGA in Verilog and C

Shelby Falde Rohit Adsule

***Abstract*—This report details the design and implementation of our COM SCI 152B final project. In this project, we designed a 2-D side-scrolling endless action platformer. The game was implemented using a Basys 3 FPGA and made use of a microblaze soft processor for game logic - allowing us to prototype and iterate in C with significantly reduced build times - while custom Verilog modules handled VGA timing, sprite rendering, and pixel mapping. Our goal was creating an immersive and exciting game that built upon various concepts we acquired throughout the course such as soft processor synthesis, custom IP block design and HW-SW co-design.**

I. Introduction

Field-programmable gate arrays (FPGAs) are primarily utilized for work-oriented applications such as signal processing and cryptographic acceleration. Their hardware-level parallelism and reprogrammability make them excellent at performing customized, low-latency tasks, yet this potential has been underexplored when it comes to recreational tasks - particularly interactive gaming - even though these same properties make FPGAs well-suited for responsive, real-time gameplay.

Our project explores this use case by implementing a 2D side-scrolling endless action platformer with gravity inversion and a projectile-based combat system. Our design extends an existing final project for COM SCI 152A created by Shelby Falde, which we re-packaged into a custom AXI4-Lite IP core. We then instantiated a MicroBlaze soft processor on top of this core. The IP core's Verilog continues to handle VGA timing, sprite and text rendering, and the final pixel pipeline, while the MicroBlaze runs the high-level game logic in C (physics, collision detection, enemy behavior, bullets, and difficulty scaling).

Processor - IP core communication proceeds via a compact set of AXI-mapped registers that use 32-bit words as "GPU buses" to carry information about the player, platforms, enemies, and bullet state. This allowed us to preserve the original hardware engine while adding new gameplay features with the much shorter edit-compile-program cycle of C.

From the user's perspective, the FPGA essentially acts as a mini-console. Most gameplay is handled by two buttons on the board (btnU and btnD, henceforth referred to as the up and down buttons respectively). The up button is used to invert gravity to travel between floor and ceiling platforms, while the down button is used to shoot enemies that would otherwise eliminate the player upon contact.

II. Related Work

Before designing our system, we surveyed both prior gravity-flip platformers and existing FPGA-based game designs that were relevant to our project. The idea of using "anti-gravity" in platformer games is not new. One of the earliest examples appears in the MSX game *Terminus* (1986) [1], where a playable character can move using reverse gravity and traverse ceilings as if they were floors. The concept came further to the fore with the NES title *Metal Storm* (1991) [2], in which gravity inversion and side-scrolling action form the core of the gameplay. Decades later, Flash and mobile games such as *Gravity Guy* (2010) revived the genre, popularizing the gravity-flip mechanic in the form of a fast, one-button endless runner and bringing it to a wide PC and smartphone audience.

In parallel, we examined the existing COM SCI 152A final project, which implemented a simpler endless-runner style game on the Basys 3. Its structure—continuous horizontal motion, obstacle avoidance, and a single primary input—was closely aligned with the behavior of *Gravity Guy*, making it a natural foundation for our design. Realizing the potential of this project when combined with a soft processor, we decided to use it as our base hardware engine. However, adapting it to a MicroBlaze-based architecture was nontrivial: online documentation for packaging a legacy game design as a custom IP core and then layering a soft processor on top were sparse, and it was initially unclear which portions of the Verilog should remain in hardware versus being moved into C.

Ultimately, we converted the original 152A design into a custom AXI4-Lite peripheral and treated it as a reusable IP block. All of the logic needed to build the scene—VGA timing, the `pixel_gen` pipeline that drew platforms, the player sprite and the text overlay—remained in Verilog. However, we extended the block diagram with several 32-bit "GPU bus" registers which MicroBlaze wrote to via our game design wrapper once per frame. These words carried packed positions, type bits, and sprite-pixel commands for the player, platforms, enemy head, bullets, and score, allowing the C program to manipulate game objects and behavior while the timing-critical rendering path remained entirely in hardware.

III. Implementation

It is important to note that this project builds upon our existing codebase from the COM SCI 152A final project. While the original implementation was in-depth, it was relatively primitive compared to the current iteration.

The original version consisted of simple rectangular platforms with semi-functional collision detection, implemented by checking if a coordinate was within specific edge ranges and coloring it white. The "Game Over" screen in the current version is a legacy artifact from this implementation, utilizing an MIT Licensed ASCII ROM in Verilog. With this current

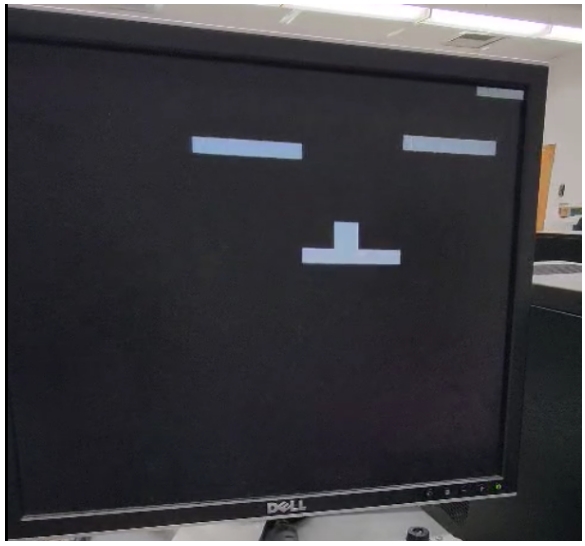

Fig. 1. The original 152A implementation featured simple white platforms and basic edge-based collision detection.

iteration, we overhauled the visuals and game logic to provide an arcade-authentic experience.

A. Hardware Overhaul

We reduced the original Verilog complexity, cutting the project down from 12 modules to 9 (excluding the IP Core wrapper). The hierarchy is shown in Fig. 2.

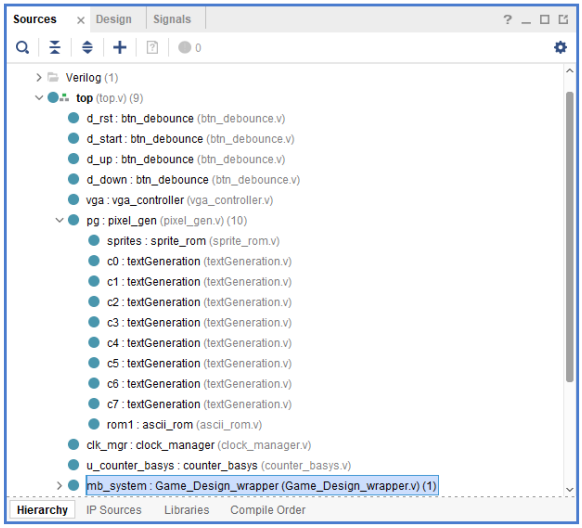

Fig. 2. The simplified module hierarchy. `top.v` drives inter-module connections, while `pixel_gen` handles texture rendering.

The three main modules are:

1) **`top.v`**: The top-level module driving wires between sub-modules.
2) **`pixel_gen.v`**: Originally responsible for 90% of game logic, this module now handles bit unpacking from registers driven by the C code and determines boundaries for texture drawing. It includes calls to `sprite_rom`, a 4096-index array (12 bits per index) functioning as a bitmap. We used a Python script with the PIL library to convert four unique 32×32 texture maps into 12-bit RGB for this black-box module.
3) **`Game_Design_wrapper`**: Connects the Verilog logic to the MicroBlaze soft processor (Fig. 5).

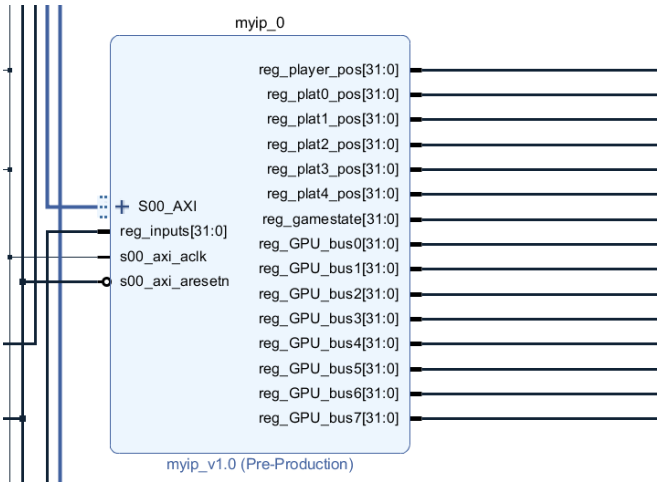

Fig. 3. The custom IP Core block diagram showing the AXI interface and custom register ports.

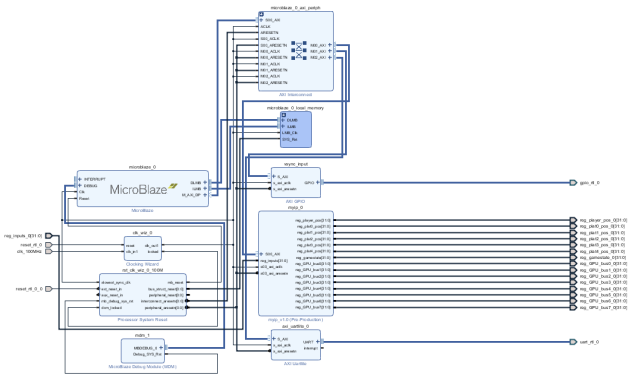

Fig. 4. The MicroBlaze block design integrating the custom Game IP, Clock Wizard, and UART.

All other major game functions were implemented via C code, connected through MicroBlaze in our custom IP core (Fig. 3). The IP core acts as a black box for a pre-designated number of registers accessible by both the C code and the Verilog wrapper.

B. Software Implementation

The most critical component is `game_platformer.c`, which contains the majority of the game logic and sprite data for the character, bullets, and enemies. We chose to drive the logic in C to adhere to our design goal of software-driven VGA logic, allowing for faster iteration without long bitstream generation times.

```
// ------------------------------------------------------------
// MicroBlaze Wrapper
// ------------------------------------------------------------
Game_Design_wrapper mb_system (
    .clk_100MHz(clk_100MHz),
    .gpio_rtl_0_tri_i(w_vsync),
    .reg_inputs_0(w_buttons),

    .reg_player_pos_0(w_reg_player_pos),
    .reg_plat0_pos_0(w_reg_plat0_pos),
    .reg_plat1_pos_0(w_reg_plat1_pos),
    .reg_plat2_pos_0(w_reg_plat2_pos),
    .reg_plat3_pos_0(w_reg_plat3_pos),
    .reg_plat4_pos_0(w_reg_plat4_pos),
    .reg_gamestate_0(w_reg_gamestate),

    .reg_GPU_bus0_0(w_reg_GPU_bus0),
    .reg_GPU_bus1_0(w_reg_GPU_bus1),
    .reg_GPU_bus2_0(w_reg_GPU_bus2),
    .reg_GPU_bus3_0(w_reg_GPU_bus3),
    .reg_GPU_bus4_0(w_reg_GPU_bus4),
    .reg_GPU_bus5_0(w_reg_GPU_bus5),
    .reg_GPU_bus6_0(w_reg_GPU_bus6),
    .reg_GPU_bus7_0(w_reg_GPU_bus7),

    .reset_rtl_0(1'b0),
    .reset_rtl_0_0(1'b1),
    .uart_rtl_0_rxd(1'b1),
    .uart_rtl_0_txd()
);
```

Fig. 5. Instantiation of the wrapper in `top.v`.

```
// ------------------------------------------------------------
// AXI register offsets (must match myip_v1_0_S00_AXI and BD wiring)
// ------------------------------------------------------------
#define REG_INPUTS       0x00
#define REG_PLAYER_POS   0x04
#define REG_PLAT0_POS    0x08
#define REG_PLAT1_POS    0x0C
#define REG_PLAT2_POS    0x10
#define REG_PLAT3_POS    0x14
#define REG_PLAT4_POS    0x18
#define REG_GAMESTATE    0x1C
#define REG_GPU_BUS0     0x20   // player sprite RAM writer
#define REG_GPU_BUS1     0x24   // enemy position/type bus
#define REG_GPU_BUS2     0x28   // enemy sprite RAM writer
#define REG_GPU_BUS3     0x2C   // bullet 0 (straight)
#define REG_GPU_BUS4     0x30   // bullet 1 (angled up)
#define REG_GPU_BUS5     0x34   // bullet 2 (angled down)
#define REG_GPU_BUS6     0x38   // 6th platform
#define REG_GPU_BUS7     0x3C   // 7-seg score bus
```

Fig. 6. AXI register offsets defined in the C header file.

1) Registers and Platform Generation: All registers used are defined in the header (Fig. 6). Notably, while there is a `reg_plat5_pos` logically, it is mapped to `REG_GPU_BUS6`. This is due to the following logic for our game that we realized a little later than expected.

We utilized an "illusion-based" approach for movement. The camera remains fixed on the character while platforms travel leftward. While the screen width is 640 pixels and platforms are 128 pixels wide ($640/128 = 5$), five platforms are insufficient. When a platform is halfway off the left side of the screen and a new one is entering from the right, the calculation becomes $64 + (4 \times 128) + 64 = 640$, requiring a 6th platform. `REG_GPU_BUS6` was added to handle this coordinate.

2) Coordinate Packing: To support "transition" platforms (platforms appearing in two rows simultaneously), we faced a register constraint. Rather than dedicating registers to 12 separate platforms, we utilized bit packing. Since platforms spawn at only 3 specific Y-coordinates, the Y-position requires only 2 bits. The X-coordinate requires $\lceil \log_2(640) \rceil = 10$ bits. This yields 12 bits per coordinate set. In a 32-bit register, we can pack two coordinate sets, optimizing our use of limited AXI registers.

3) Dynamic Entity Generation: In order to fully exploit the reprogrammability of C, we went with an unconventional approach for drawing the dynamic actors; the player sprite, the enemy sprite, and the three bullets fired by the player. All of these were driven from the C code through a small set of 32-bit "GPU bus" registers exposed by the custom AXI4-Lite IP. We created these ports by adding them to the myip block design files `myip_v1_0_S00_AXI.v` and `myip_v1_0.v` and then using the block diagram to make each port external, wiring it to a 32-bit register of the same name.

In the block diagram these appear just below the platform registers at offsets `REG_GPU_BUS0` – `REG_GPU_BUS7`, and are then wired from the top-level module into `pixel_gen.v` via the game design wrapper. Each bus acts as a tiny command port into on-chip BRAM or into simple drawing logic inside `pixel_gen`.

The player sprite is stored in a 32×32 BRAM (`player_ram`) inside `pixel_gen`. On the Verilog side, `reg_GPU_bus0` is decoded as:

- bit 31 – write-enable edge
- bits [21:17] – Y index
- bits [16:12] – X index
- bits [11:0] – 12-bit RGB color

On every rising edge of the write-enable bit, `pixel_gen` writes that color into `player_ramy,x`. The C helper `gpu_write_pixel()` constructs this 32-bit word and sends it over AXI with:

```
u32 packet = (1u << 31)
           | ((y & 0x1F) << 17)
           | ((x & 0x1F) << 12)
           | (color & 0x0FFF);
Xil_Out32(GAME_IP_BASEADDR + REG_GPU_BUS0, packet)
Xil_Out32(GAME_IP_BASEADDR + REG_GPU_BUS0, 0);
```

`load_robot_sprite()` (see Fig. 7) then walks over a small 2-D C array describing the robot/astronaut and calls `gpu_write_pixel()` to paint it into a 32×32 tile. At render time, `pixel_gen` treats this BRAM as a sprite sheet: it checks whether the current VGA `(x,y)` lies inside the 128×128 player box, scales the coordinates down by 4, and samples `player_ram`. This lets us redesign the main character entirely in C without touching the Verilog or re-synthesizing the bitstream.

The enemy head is handled in the same way using a second BRAM (`wyvern_ram`) driven by `reg_GPU_bus2`. The C functions `enemy_draw_base_head()` and `enemy_draw_mouth(state)` program this RAM via another helper, `gpu_write_enemy_pixel()`, to draw the green face, horns, eyes, and mouth. An animation state machine in `enemy_anim_tick()` runs once per frame (on

```c
void load_robot_sprite() {
    const u8 sprite[10][12] = {
        {0,0,0,0,3,3,0,0,0,0,0,0},
        {0,0,0,1,1,1,1,0,0,0,0,0},
        {0,0,1,1,2,2,1,1,0,0,0,0},
        {0,0,1,1,1,1,1,1,0,0,0,0},
        {0,1,1,1,1,1,1,1,1,0,0,0},
        {1,1,3,1,1,1,1,3,1,1,0,0},
        {1,0,3,3,3,3,3,3,0,1,0,0},
        {0,0,1,1,0,0,1,1,0,0,0,0},
        {0,0,1,1,0,0,1,1,0,0,0,0},
        {0,3,3,0,0,0,0,3,3,0,0,0}
    };
    u16 c_white = 0xFFF;
    u16 c_red   = 0xF00;
    u16 c_grey  = 0x888;
    int off_x = 10;
    int off_y = 11;

    for (int y = 0; y < 32; y++)
        for (int x = 0; x < 32; x++)
            gpu_write_pixel(x, y, 0x000);

    for (int r = 0; r < 10; r++) {
        for (int c = 0; c < 12; c++) {
            u8 val = sprite[r][c];
            if (val == 0) continue;
            u16 color = (val == 1) ? c_white : (val == 2) ? c_red : c_grey;
            gpu_write_pixel(off_x + c, off_y + r, color);
        }
    }
}
```

Fig. 7. Drawing of robot sprite via C code.

VSYNC), cycling through "closed," "half-open," "wide-open," and "clamp" states. Each transition simply redraws the eyes and mouth region by issuing a new batch of GPU-bus writes. From the hardware's perspective, nothing changes except the contents of BRAM; the rendering path in `pixel_gen` just samples `wyvern_ram` at the appropriate `(x,y)` whenever the current pixel lies inside the 32×32 enemy box.

Bullets are slightly different: they do not use a sprite RAM, but instead are drawn as small filled rectangles whose positions are supplied from C. For each of the three bullets we pack `x`, `y`, and a 4-bit "type/active" field into a 32-bit word using `pack_pos_and_type()` and write these to `REG_GPU_BUS3`, `REG_GPU_BUS4`, and `REG_GPU_BUS5`. In `pixel_gen`, the corresponding input registers are unpacked into `bullet_x`, `bullet_y`, and `bullet_type`; if `bullet_type` $\neq 0$ and the current `(x,y)` lies within an 8×4 region starting at `(bullet_x, bullet_y)`, the pixel is colored space blue. This scheme lets the C code control up to three independently moving bullets by updating just three 32-bit registers per frame.

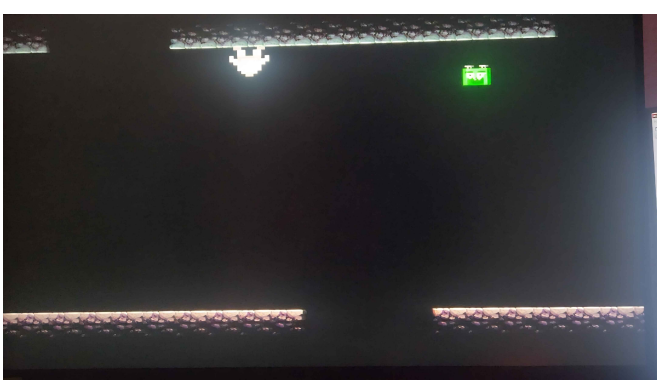

Fig. 8. Gameplay screenshot showing transition platform columns mixed with regular single platform columns, featuring the character and an enemy on screen.

4) Synchronization and Collision: To synchronize operations with the 60Hz monitor refresh rate, we read the VSYNC variable from the AXI GPIO using `XGpio_DiscreteRead(&vsync_gpio, 1)`. This signal flips upon every display refresh, allowing accurate timing for game object updates.

For collision detection, we implemented Axis-Aligned Bounding Box (AABB) detection in a single function, which is efficient for rectangular hitboxes (Fig. 9).

```c
// ============================================================
// ENEMY & COLLISION HELPERS
// ============================================================
static int rect_overlap(int ax, int ay, int aw, int ah,
                        int bx, int by, int bw, int bh) {
    return (ax < bx + bw &&
            ax + aw > bx &&
            ay < by + bh &&
            ay + ah > by);
}
```

Fig. 9. Axis-Aligned Bounding Box (AABB) collision logic implemented in C.

C. Thematic Note

Although the source code refers to the enemy as a 'wyvern' and the player character as a 'robot,' playtesters interpreted the sprites as aliens and a rocket ship. This unintended sci-fi aesthetic proved to be a better thematic fit considering the gravity mechanics, inspiring the title *Space Runner*.

IV. Results

The result of our project *Space Runner* was a fully functional side-scrolling action platformer which ran as intended. The project was play-tested and received positive feedback from all those who tested it. A myriad number of future modifications can be considered, including but not limited to: gamifying the prototype by adding discreet level markers for a more cohesive experience, altering platform design by adding interactive blocks such as a speed block which would accelerate the user when they step on it, and adding new mobile enemies which follow the user's movement and shoot at them from behind.

V. Conclusion

Our *Space Runner* game showcases the potential of running an arcade-style game on hardware as resource-constrained as an FPGA board. It demonstrates the usage of sprite drawing via VGA through a hybrid approach, utilizing both Verilog for rendering and C for logic via custom IP Core integration. We have gained valuable insights into programming as a whole and optimizing memory usage given tight bounds, which is generally very important in subjects like embedded systems. Overall, this project advances the progress—not often fully explored in introductory designs—of creating an in-depth game with detailed textures and color mappings in the world of FPGAs.

References

[1] Mastertronic. (1986) Terminus (1986) - msx gameplay [video]. Accessed: Dec. 2025. [Online]. Available: https://youtu.be/l8uXO71fBGA?t=717

[2] Irem. (1991) Metal storm (1991) - nes gameplay. [Video file]. Accessed: Dec. 2025. [Online]. Available: https://youtu.be/ZLUc7KJteW0

Project Download:

Project Download Link from Shelby Falde's
Google Drive